

Exploratory Programming:
 Build and Fix
 Quick development
 Small ✓ Large X
 Knight Capital Group
 → Financial Critical System

SDLC: (Perning's Cycle)
 1) Feasibility Study (Planning, Estimation, Tracking)
 2) Req. Analysis & specs. (Documentation)
 3) Design
 4) Coding
 5) Testing & Deployment (Qs/QT)
 6) Maintenance

(Inspection) Static Testing
 (Unit, Intg., ...) Dynamic Testing
 Design (ISY) Req. - Testing - Code; (40%) Unit testing
 Acceptance Testing Design
 High Level Low Level (25%) Testing (Low-level) System (20%) Testing (High-level)
 Maintenance

Teamwork
 Communication
 Programming Process
 SRS (Blackbox) Process Model
 Convergence
 Upfront Planning
 Under-estimation
 Leadership
 Psychological Safety
 COBOL

Models
 Traditional (Plan Driven)
 Agile (Rapid Changes) (Scrum) (Kanban)
PDCA: → PDSA
 Check (Testing & Deployment)
 Do (Implementation)
 Plan (Planning, Estimation, Req., Design)
 Act (Retrospective) study

Code
 Frontend Backend
 Programming Language
 (PEP-8) (CERT - C/C++)
Model Driven Development:
 Writing without code
 Feasibility Study
 Technical Schedule Expensive
 Operational Legal
 (Planning, Estimation, Tracking, Artifacts)

Preventive (Change to prevent error)
 Corrective (Correct errors which got mixed [Bugs])
 Perfective (Improve & Enhance)
 Adaptive (Porting to new)
 (Elicitation, Analysis, Negotiation, Req. Dev., Documentation, Val.)
 Requirements Engineering (PRD → SRS)
 Functional (What to do) Technical Business
 Non-functional (Qualities) (How good system must be)
 Marketing
 Verification: Am I building the product right
 Validation: Am I building the right product
 MosCOW (Must, Should, Could, Won't)

Green Field
 Brown Field
 (Test) Conformance
 Work-in-Progress
 Definition-of-Done
 Real-World Parallel
 Sustainable Pace
 Harder vs. Smarter
 The Retrospective
 Technical Debt. (Rework)
 User Story
 Refactoring - Modularize (Dead Code)
 Open-Closed Principle
 DevOps
 Test Driven Development
 Continuous Integration
 Scope: System doing test
 Scope Creep (Clear Funct.)
 Low Fidelity: Figma
 Joint Appl. Developer

Req./Planning → 30-40%
 Proj. Mgmt./Est. → 20-30%
 People & Org → 15-25%
 Tech./Design → 10-20%
 Testing/QA → 5-15%
 Change Mgmt

Design
 High level Software Architecture (Artifact, Design, Document)
 Low level Detailed Design (Interface) Document → UML

Traditional Models
 Waterfall Process (Sequential Documentation Req.: Stable Static)
 Prototyping Process (Req.: Unclear Focus on Frontend Users)
 Evolutionary Process (Iterative Incremental of Waterfall)
 Spiral (Planning, Eval., Identify & resolve risks, Engg.)
 (Meta-Model Safety-Critical less Budget)
 Speed ↑ large Project

Reg. Req. ↔ Acceptance Testing
 High Level Design ↔ System Testing
 Low-level Design ↔ Integration Testing
 Implementation ↔ Unit Testing (Module design)
 KANO (Basic Differentiator, Delighters)
 (Arch)

Role: Prod. Owner, Dev. Team
 Artifact: Product Backlog
 Daily Scrum (Sprint)
 Split Dev. Work
 Dev Work

Designing
 Coding
 Manage OS Database
 Operations
 Monitor load spikes
 Performance crashes
 Backups Rollback releases
 HW updates
 CI
 Testing
 Severity: Impact on system
 Priority: Urgency for fixing (Blocker, Critical, Major, Minor, Trivial)
 Req.: Statement that specifies what a system must do or qualities it must possess to satisfy what stakeholder needs
 [Bug: When Req. don't work]

Defect Lifecycle
 New → Rejected
 Open → Deferred
 Fixed
 Retest → Reopen
 Closed
 Correct Feasible
 [Trace] Necessary Prioritized
 Validation
 Complete
 Consistent
 Unambiguous
 verification
 Concise
 Traceable
 Non-Redundant
 Organized
 Conformant
 Verifiable
 Bad Req.
 Inconsistent - Contradict
 Incomplete - Omitted
 No numbers - Approx.

Agile Models
 User Stories
 Refactoring
 Continuous Integration
 Test-Driven Development
 Team Skills
 Communication/Shared Ownership
 Small teams
 Big Bang
 Commit → Test → Build
 Feedback
 Compile checker
 (D): DevOps
 Manual → Delivery
 Automatic → Deployment

Req. Gathering → ID, Title, Descr., Priority, Type, Source, Acc. Criteria, Notes (F/NF) (stakeholder)

Usability
 Complexity
 Auditability
 Capacity
 Configurability
 - Internationalization
 - Personalization
 - Variability
 Correctness
 - Accuracy
 - Precision
 - Current
 Efficiency
 Interoperability
 Performance
 - Response Time
 - Latency
 - Throughput
 - Schedulability
 Dependability
 - Availability
 - Reliability
 - Robustness
 - Safety
 - Security
 - Survivability

SRS:
 Field
 System
 ID
 Title
 Statement
 Preconditions
 Postconditions
 Priority
 Acc. Criteria
 Notes
 Upon - Where As - so that
 Actor (Trigger behaviour [n. vs. Sa.])
 Sub Use Case
 Super Use Case
 system

Use-case: → (Nouns)
 No. Name Overview Type Actors Properties Constraints
 Precondition Flow - Main & Alt. Post-condition Cross ref.
 FT type
 Transactional FT Data FT
 Input Output Inquiries Logical interface External interface
 Multipliers
 UML

I. Aggr. ind. est.
 II. Range, Not No.
 III. Why estimate?
 IV. Est. ≠ Commitment
 V. Measure, count ✓ est. X
 FPA
 - count no. of fn. per category
 - complexity factor
 - VAF Multiplier
 - FP Total
 - Calc time

External Inputs: Input Data
External Outputs: Outputs sent out
External Enquiries: Queries that retrieve data with less processing
Internal logic files: App.-maintained logical data groups
External Interface files: Read only logical data used by app maintained externally

Complexities - 3, 4, 6 (Input)
4, 5, 7 (Output)
3, 4, 6 (Enquiries)
7, 10, 15 (ILF)
5, 7, 10 (EIF)
VAF = 1.20
FPA → 143 FP ← 2860 hr / 20 hr per FP
5 devs ← 150 hr / week
Timeline → 2860 / 150 ≈ 19 weeks
Buffer ≈ 20% ⇒ 23 weeks

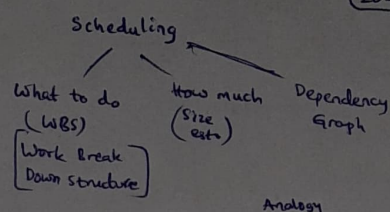
VAF = 1.20

① Developer Testing
Exhaustive Testing
Random Testing
Equivalence Classes
Coverage - Statement
Condition
Decision
Branch
MCDC - Safety critical
(Modified Condition Decision)

Black box Testing - System level (Req.)
Whitebox Testing - All levels (Code)
Model-based Mutation (small change)
Combinatorial Path

Error - Human mistake
Fault - Defect (Flaw)
Failure - System dev.
ISTQB
(1) Testing shows presence of defects
(2) Exhaustive Testing: Impossible (risk-based selection)
(3) Early testing saves time and cost
(4) Defects clusters in few modules
(5) Pesticide Paradox - Upgrade test new bugs
(6) Testing depends on context (Safety, critical vs. web app)
(7) Absence of errors finding defects ≠ success

Needs
↓
Features
↓
Use-cases
↓
Scenarios
↓
Test Cases
White-box Testing
- Coverage
- Statement
- Branch
- Condition
- Dataflow
- Mutation
- Security
- Maintainability



WBS
Process: Reg., Analysis, Design, Testing, PM
Product: Interface, DB, Engg. mgr.
Hybrid: Kanban Board, Backlog, In Progress, Done

Earned Value Management:
Planned Value
Earned Value
Actual Cost
EV (CPI, SPI)

Performance Testing (Response Time, Throughput)
Load Testing (Normal workload)
Stress testing (Beyond normal)
Endurance Testing (Long-duration stability)
Spike Testing (Sudden load changes)
Test Case Template
- Identifier
- Objective
- Priority
- Traceability
- Precondition
- Inputs
- Expected Results

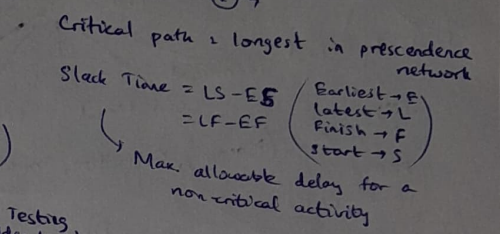
Unit-Testing - Stubbing
Understand Req.
Complex Logic
Dependencies (tab)
State Mgmt.
Test maintainability
Integration Testing: Big-bang, bottom-up, Top-down, Critical first

Testing Process
Planning
↓
Design
↓
Execution
↓
Evaluation
↓
Closure
Coverage: Function (each method), Statement (each line), Path (through code)

Dimensions of Testing
Quality under Test
- Performance
- Usability
- Scalability
- Portability
- Security
- Maintainability
Unit under Test
- Manual
- Automatic
Type of Test
- Whitebox
- Blackbox
Dataflow Testing (LHS) Defining Node (in), (RHS) Usage Node (out)
Metrics (Jenkins)
Test: Coverage, DRE
Process: Cycle time, MTTR, MTBF (Repair) (Failure)

Scheduling
Gantt Charts
Easy but Vague (No interdependency)
Network Tech.
CPM (Critical Path Method)
PERT (Program Eval. & Review Tech.)

Task	Duration	Dep.	ES	EF	LS	LF
A	9	start	0	9	0	9
B	5	A	9	14	11	16
C	7	A	9	16	9	16
D	11	A, C	16	27	16	27



Critical path = longest in precedence network
Slack Time = LS - ES
= LF - EF
Max. allowable delay for a non-critical activity

System Testing
Real world env.
Speed, latency, load
User & Designer
Last place of faults
Acceptance Testing
Alpha Testing - Supervision
Beta Testing - Normal env.
Crash
Compliance
Field
Release or not (Shadowing)

Test Plan ⇒ log all defects ⇒ Document
Inspection ⇒ Code Review, Pair prog., formal inspection, Checklists, (SonarQube)
Whitebox Testing - Loop Boundary Analysis, Data Dependency, Path Predicate, combinatorial, Error Queueing, Mutation, Model based, Static program analysis
Blackbox Testing - Equivalence Partitioning - divide into valid/invalid classes
- Boundary value Analysis - Edge values
- Decision Table testing - complex business rules
- State transition testing - val. behaviour across states
Regression Testing: functional; Funcs work after changes

Equivalence classes - Groups treated same by Algo

Decision Table:

	Rule 1	Rule 2	...
Conditions			
Condition-1	Yes/No	Yes/No	
Actions			
Action-1	x	y	

State transition state
Set of Testcases
All states → visited atleast 1
All events → Triggered atleast once
All paths → executed atleast once
All transition → exercised atleast once
Path Testing: V(G) = no. of paths
Mcabe's cyclomatic complexity
no. of simple decisions + 1
no. of enclosed areas + 1
[Decisions → Binary]
C = edges - nodes + 1

NF Qualities:
Performance
Scalability
Security
Usability
Reliability/Availability
Maintainability